

Climate Mesh

A decentralised, explainable climate early-warning mesh on a Raspberry Pi 5

Luis Yu and Leo Zhang · Years 12–13

PA Raspberry Pi Competition 2026/27 · Theme: Building a Positive Human Future (Safer Societies & Sustainable World)



1. The problem

The official river gauge nearest to a school or an estate can be kilometres away from the stream, drain or underpass that actually floods. By the time an official warning reaches a community, the water is already rising. The World Meteorological Organization reports that countries with limited early-warning coverage suffer nearly six times the disaster mortality of countries with substantial coverage (4.05 versus 0.71 deaths per 100,000 people; WMO/UNDRR, *Global Status of Multi-Hazard Early Warning Systems*, 2023). There is a gap between where climate events happen and where they are measured.

We picked this because it is the rare climate problem two students can actually build something about. Reading about early-warning systems, what struck us was that the hard part is not the science but the last mile: the forecast exists, and nobody is watching the particular street. A watcher for one street is software and an £85 computer, which is exactly what we had.

We wanted a low-cost way for a school or a community to watch its own streets, and to be told in plain English what to do.

2. What we built

Climate Mesh is a network of 20 environmental “nodes” across Greater London, simulated or fed by live weather data, that runs on a single Raspberry Pi 5, fully offline, with no cloud account and no API key. Every two seconds each node reports eight channels: temperature, humidity, air quality, water level, wind speed, wind chill, heat index and barometric pressure. A risk engine scores every location from 0 to 100, an explainable AI model flags unusual combinations of readings, neighbouring nodes have to agree before a risk is escalated, and plain-English alerts with community action playbooks appear on a seven-tab dashboard.

3. How it works

One reading shape for every source. Whether a value comes from a physical sensor, the live Open-Meteo weather service or our simulator, it is turned into the same canonical reading: the eight channels plus a `source` label (`hardware` / `api` / `demo` / `simulation`) and a `quality_flag` (`ok` / `estimated` / `stale` / `missing`). Everything downstream (the risk engine, the AI, the database, the dashboard and the evidence export) only ever sees that shape. A real sensor joins the mesh by adding one adapter; nothing else changes.

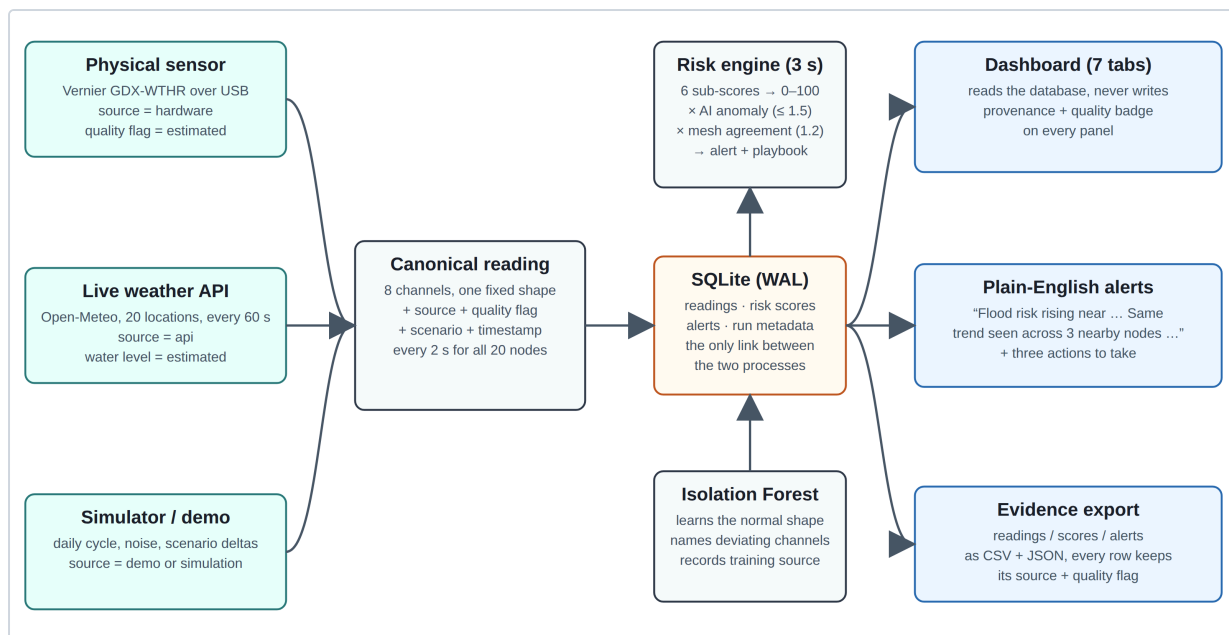


Figure 1 — The data path. Every source emits the same canonical reading (eight channels plus source and quality flag), so a real sensor is a drop-in and nothing downstream changes; one small database file is the only link between the engine and the dashboard.

An explainable risk score. For each node the engine computes six hazard sub-scores on a 0–100 scale (temperature, humidity, air quality, water level, wind, pressure); wind chill and heat index are derived comfort values (NOAA/Steadman) shown to the user, not scored twice. The sub-scores are combined as *worst hazard plus 20 % of the rest*, so one severe hazard alone can reach CRITICAL while several moderate ones also escalate. The bands are SAFE (under 30), MODERATE (30–60), WARNING (60–80) and CRITICAL (80–100). Temperature and humidity are two-sided: a frosty morning is filed as a *cold* hazard with its own advice, never as a “heatwave”.

AI that explains itself. An Isolation Forest learns the normal shape of the data and flags readings that are easy to isolate (a bit like spotting the odd one out in a crowd: a reading that can be separated from all the others in a few cuts is unusual), so an unusual *combination* of values is caught while each channel is still only moderately elevated. A confirmed anomaly multiplies the node’s graded risk by up to $1.5\times$ (measured range $1.25\times$ to $1.36\times$), and the model reports which channels deviate most from the learned baseline, so every alert says *why*. Deliberately, the AI can amplify risk but never create it, and the bound is arithmetic rather than anecdote: the AI and mesh multipliers compose to at most $1.5 \times 1.2 = 1.8$, so a node whose own sub-scores are below $33.4/100$ can never reach WARNING and one below 44.5 can never reach CRITICAL, whatever the two layers say. Measured over 80,000 synthetic readings the AI multiplier actually lands between $1.25\times$ and $1.36\times$ against that 1.5 design ceiling. A test sweeps the whole range and fails if the bound is ever exceeded. Offline it trains on 2,000 deterministic synthetic samples; when online it can instead train on about 30 days of real hourly ERA5 weather and air quality for a representative central-London point from the Open-Meteo archive (cached after the first fetch), and it records which path it used. Every number in this write-up was produced with the synthetic path; training on the archive is a one-flag change (`--ai-training historical`).

Neighbours have to agree, both ways. Two nodes within 6 km are neighbours, and agreement cuts in both directions. A node whose neighbours see the same hazard is escalated (1.2×). A node that is elevated while *none* of its neighbours sees anything is damped instead (0.75×) and is not reported as the hazard at all: it raises a *sensor-check* alert telling the site team to go and look at that node, because one instrument disagreeing with four neighbours is far more likely to be broken than to be right. That is the difference between a claim and a mechanism: drop a stuck water-level sensor reading 9 m into an otherwise calm mesh and the old rule gave CRITICAL 100/100 and a flood playbook; the rule as built gives 75/100 and “check the sensor at Regent’s Canal ... none of its 4 neighbours within 6 km sees the same thing”. Make two of those neighbours agree and the same reading becomes a CRITICAL flood alert. A node with fewer than two neighbours in range cannot be corroborated either way, so it is left alone and the dashboard says so rather than pretending. A worked example from the flood frame: Hyde Park’s own sub-scores give a base of 58.0 (MODERATE). Four of its neighbours are elevated for the same flood hazard, so the mesh multiplier applies: $58.0 \times 1.2 = 69.6$, WARNING, and an alert is raised. Regent’s Canal, by contrast, saturates on its own: water 83.5 + 20 % of (humidity 67.8 + pressure 37.3) = 104.5, capped at 100, before any multiplier. In the heatwave frame Lewisham shows both layers at once: base 65.8 (WARNING) × AI 1.26 × mesh 1.2 = 99.5, CRITICAL.

Alerts people can act on. WARNING and CRITICAL results raise an alert containing the plain-English explanation and a playbook of practical, low-risk actions for that hazard (flood, heatwave, smog, storm, cold). Alerts are rate-limited so the log never fills with duplicates.



Figure 2 — Node Detail, Regent’s Canal, flood scenario, demo data: the “Why:” sentence, what the mesh could vouch for, the actions, and the six sub-scores behind a 100/100 score (this node saturates on its base score; see the Hyde Park example above for the mesh layer at work).

How the parts talk. The engine writes every reading and score into one small database file on the Pi; the dashboard only ever reads that file, so it can never alter the data, and either half can be restarted without losing anything. The sensor loop reads all 20 nodes every 2 seconds (every 60 seconds in `api`

mode, to respect Open-Meteo’s free service) and the risk engine scores them every 3 seconds. A one-command export writes every reading, score and alert to CSV and JSON, each row still carrying its source and quality flag, plus a table of how many readings came from each source.

4. What we think is new

Not any single part, but the combination:

- **Neighbours as a two-sided trust signal.** Agreement escalates a cheap node (1.2×) and *disagreement damps it* (0.75×): a node alone in seeing a hazard is reported as a sensor to check, not as the hazard, so a network of £85 nodes is harder to fool than one expensive sensor. We test both directions, including the case where the mesh must refuse to escalate.
- **A bounded, explainable AI layer.** The Isolation Forest names the channels responsible and can only amplify graded risk, never create it. The two layers compose to at most 1.8×, so nothing below 33.4/100 on its own sub-scores can be pushed to an alert.
- **Provenance enforced in code, not by convention.** Every reading, panel, alert and exported row carries its source and quality flag; `is_simulated` is derived from `source` so the two can never disagree; a reading with an unknown source or flag is rejected at construction; a missing or non-numeric channel scores 0 rather than falling through to CRITICAL; each score is rounded once so the stored number, its band and the alert text always agree; and the dashboard can only show a “Physical Sensor” badge when a reading really came from a device.
- **Reproducible by a stranger.** The whole pipeline, model included, runs offline on one Pi 5, and one command reruns our entire evidence chain.

5. What is real and what is simulated

We label exactly what this is and is not.

Mode	Data	Internet	Sensors
<code>simulation</code>	Realistic generated data (default)	No	No
<code>demo</code>	Deterministic, screenshot-stable data	No	No
<code>api</code>	Live Open-Meteo weather and air quality for all 20 locations	Yes, falls back to simulation	No
<code>hardware</code>	One physical Vernier weather sensor, simulated mesh for the rest	No	Yes, falls back if absent
<code>auto</code>	Hardware, else API, else simulation	Optional	Optional



Figure 3 — Live Map, flood scenario, demo (simulated) data, on the dashboard's offline basemap. Each circle is one of the 20 nodes; colour and size show risk 0–100; the blue lines are the Thames, the Lea, the Wandle and Regent's Canal. The river and canal nodes are the ones in red.

- The 20 nodes sit at well-known London landmarks chosen to exercise the environment types (river, residential, urban, park). They are illustrative, not deployment sites.
- The hardware path (a Vernier Go Direct Weather sensor, GDX-WTHR, over USB) is implemented and tested end to end with a stand-in device object that mimics Vernier's helper library (it opens, answers, answers badly or refuses to open, and the adapter is checked for each), but it has **not yet been validated against a physical device**. A node is labelled `hardware` only after a device actually opens and returns a reading.
- The GDX-WTHR senses temperature, humidity, wind and pressure only. In `hardware` mode the air-quality and water-level channels are conservative placeholders, so a hardware reading is always flagged `estimated`, never `ok`: a `hardware/ok` label can never cover a channel that was not measured.
- In `api` mode the water level is derived from precipitation, not read from a river gauge, so it is flagged `estimated`; it is an indicator, not a measurement.
- The flood, heatwave, smog and storm scenarios are simulated so that judging never depends on a real emergency. In `api` and `hardware` modes live values are shown unmodified.
- One node (Ilford) has a single neighbour inside the 6 km radius, so the corroboration layer can never fire for it in either direction. The engine marks it `unavailable` rather than silently treating it as agreement, and the dashboard says “cannot be corroborated” next to its score.
- The AI is decision support for a community, not an accredited warning system. Alerts always defer to official guidance.

6. Evidence that it works

- **212 automated tests pass** on Python 3.11, 3.12 and 3.13, including a test that executes all seven dashboard tabs in Streamlit's headless test harness and tests that pin the exact numbers quoted below. Continuous integration runs the suite on every push to `main` and every pull request, on x86-64 (Python 3.11 and 3.13) and on a 64-bit Arm Linux runner (Python 3.11), the Pi's architecture; 3.12 was checked by hand.
- `python scripts/judge_validate.py` runs the smoke test, all 212 tests, a normal and a flood demo cycle and the evidence export → **PASS (5/5 steps passed)**. Its full output is `docs/screenshots/terminal-judge_validate.png` in the repository.
- `python scripts/demo_tour.py` runs one deterministic cycle of all five scenarios in a few seconds. Repeated runs print identical numbers; its full output is reproduced in the Appendix.
- **The mesh claim, tested rather than asserted.** Injecting one stuck water-level sensor (9 m) into an otherwise calm mesh produces **0 flood alerts and 1 sensor-check**, at 75/100 rather than 100/100. Making two of that node's neighbours report the same thing turns the identical reading into a CRITICAL flood alert. Six tests in `tests/test_corroboration.py` pin both directions, including the node that cannot be corroborated at all.

Do the layers earn their place? Counting how many nodes changed band is activity, not accuracy, so `python scripts/evaluate.py` measures the three things that matter, on 1,290 independently seeded frames. Each configuration sees identical frames; a frame counts as detected when some node raises an alert for the hazard that was actually injected.

Configuration	False alarms, 30 quiet frames	False flood alerts, 90 frames with one stuck sensor	Real events detected, full intensity
Sub-scores only (thresholds)	0 %	100 %	100 %
+ AI	0 %	100 %	100 %
+ AI + mesh (shipped)	0 %	0 %	100 %

The middle column is the whole argument. Given one jammed water sensor and nothing else happening, thresholds alone raise a false flood alert every single time, and so does the AI, because neither has any way to ask whether anyone else can see it. The shipped system raised none, and a sensor-check instead — without giving up a single real detection at full intensity.

The honest cost is in between: across intensities 0.5 to 0.7, where an event is real but only half-formed, the mesh detects 39–63 % of frames against 50–75 % for thresholds alone, because it holds back nodes no neighbour has confirmed yet. We would rather be a step late on a half-formed event than cry wolf every time an instrument fails, and the numbers say which we bought.

The demo tour, one row per scenario:

Scenario	Average risk	Worst node	Level	Alerts
normal	3.9	Greenwich	SAFE	0
flood	50.8	Regent's Canal (Little Venice)	CRITICAL	10
heatwave	78.8	Brixton	CRITICAL	12
smog	82.7	Brixton	CRITICAL	15
storm	98.1	Brixton	CRITICAL	20

What the AI and mesh layers add, on the same deterministic frames (number of nodes in each band):

Scenario	Sub-scores only	With AI and mesh	What changed
normal	20 SAFE	20 SAFE	Nothing: no node is flagged anomalous, no false alarm.
flood	4 CRITICAL, 8 MODERATE, 8 SAFE	4 CRITICAL, 6 WARNING, 2 MODERATE, 8 SAFE	Neighbour agreement lifts six residential and park nodes to WARNING; alerts 4 → 10.
heatwave	8 CRITICAL, 4 WARNING, 8 MODERATE	11 CRITICAL, 1 WARNING, 8 MODERATE	The AI flags 11 nodes as anomalous (eight urban, three residential); the three residential nodes move from WARNING to CRITICAL.
smog	12 CRITICAL, 8 MODERATE	12 CRITICAL, 3 WARNING, 5 MODERATE	Neighbour agreement lifts three river and park nodes; alerts 12 → 15.
storm	5 CRITICAL, 15 WARNING	19 CRITICAL, 1 WARNING	18 AI anomalies plus agreement across 19 nodes.

The layers only ever amplify a hazard the sub-scores already see; they cannot invent one. These counts are pinned by a test, like the other numbers in this section.

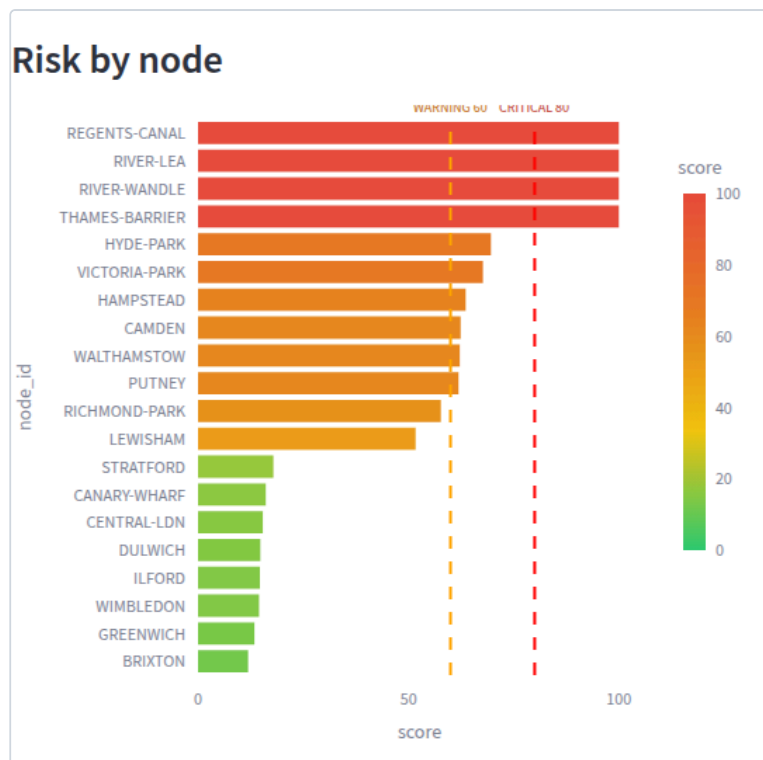


Figure 4 — Network Overview, flood scenario, demo data. The four river and canal nodes go CRITICAL while inland nodes such as Brixton and Greenwich stay SAFE: the hazard lands where it should. The dashed lines are the WARNING and CRITICAL thresholds, drawn on the chart so a reader does not have to know them.

Reading it cold

Everything above is a measurement, and none of it answers a different question: can somebody who did not build this look at the screen and know what to do?

So we walked through the dashboard as three readers who are not us — the person who deals with a flooded entrance, the person who knows what the data means, and the person who knows how the code works — and set one task, with no explaining allowed: *there is a flood warning somewhere in this system; find it, and say what you would do.*

Finding it took seconds. The second half failed outright. Every alert the engine raises has carried a plain-English action playbook from the start — clear the drains, inspect low-lying entrances, review the evacuation route — and the engine writes that playbook into the database with every alert we have ever raised. **The dashboard never displayed it.** Somebody who found the red node had nowhere to go — and §2 of this document, written before we looked, says those playbooks appear on the dashboard. They did not.

The same reading found three more. The only thing saying whether 62/100 was bad was the colour of a dot — exactly the cue a red-green colour-blind reader does not get. The sidebar called a node REGENTS-CANAL while the map beside it called the same node Regent's Canal. And nothing on the page gave the time of the readings, so a live dashboard and one left open since yesterday looked identical.

All four are fixed. The first screen now leads with the action (Figure 5): the worst node, its band in words, why it scored what it did, whether the mesh can vouch for it, and the numbered steps. Every score carries its band as a word as well as a colour, the sidebar uses the map's names, and the header shows the age of the newest reading and says so plainly once it is more than two minutes old. Twenty tests hold this in place, including one that fails if the playbook stops being displayed and one that checks a single stuck sensor still sends the reader to the equipment rather than to the river.

We should be straight about what this was. It was us reading our own work as carefully as we could; it was not a stranger using the system. Putting it in front of somebody who actually manages a building, and changing it again based on what they do rather than what we predict they would do, is the most valuable thing left on our list.

● Act now — Regent's Canal · 100/100 CRITICAL · flood

Why: Flood risk rising near Regent's Canal (Little Venice). Same trend seen across 3 nearby nodes. Risk score 100/100. Main contributors: critical water level (4.8 m), very high humidity (99%), falling pressure (996 hPa).

✓ **Confirmed by the mesh** — 3 of this node's 4 neighbours within 6 km report the same hazard.

What to do

1. Check and clear nearby drains and gullies.
2. Inspect low-lying paths and entrances for standing water.
3. Review the evacuation route and move valuables off the ground floor.

Also above the action threshold: River Lea (100 CRITICAL), Thames Barrier (100 CRITICAL), River Wandle (100 CRITICAL), Hyde Park (70 WARNING), Victoria Park (68 WARNING), and 4 more. Open **Node Detail** for any node's full breakdown.

Figure 5 — The first thing on the Live Map, flood scenario, demo data. Before this walkthrough the screen showed the map alone: the action playbook existed, was attached to every alert, and was never displayed. The mesh line above the steps is the corroboration state — here three of four neighbours agree, so this is a flood; with none agreeing it reads as a sensor to check instead.

7. The Raspberry Pi 5

The Pi 5 is the whole product: it runs the simulator or the live data source, the scikit-learn model, the risk engine, the database and the web dashboard at the same time, headless, at the site.

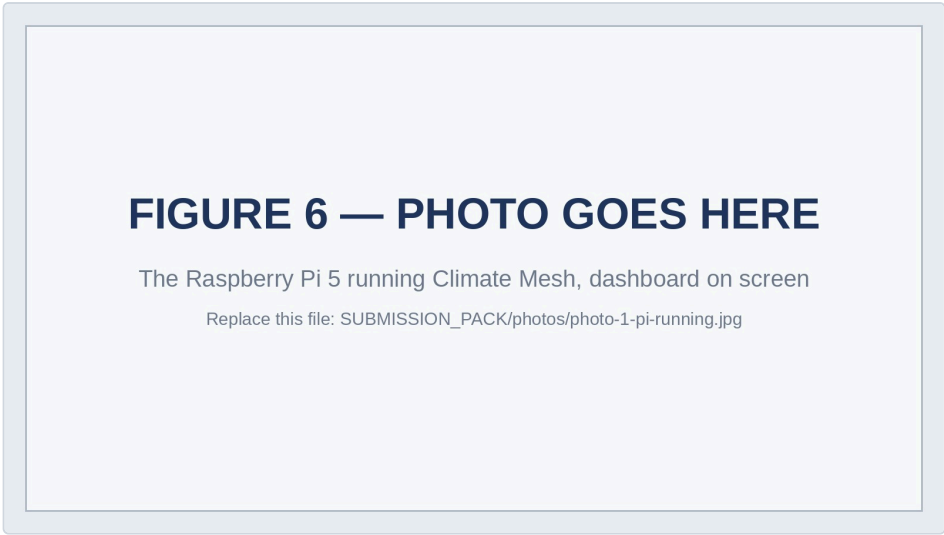


Figure 6 — ► PLACEHOLDER: replace photos/photo-1-pi-running.jpg with a photo of the Raspberry Pi 5 running Climate Mesh, and change the picture in WRITEUP.docx. Caption to use: “The Raspberry Pi 5 running Climate Mesh.”

Bill of materials for one node (approximate UK prices, September 2026):

Item	Purpose	Approx. cost
Raspberry Pi 5 (4 GB)	Runs everything	£55
27 W USB-C power supply	Power	£12
32 GB microSD card	OS and software	£8
Case with fan	Protection	£10
Core node total		£85
Vernier Go Direct Weather (GDX-WTHR), optional	Physical temperature, humidity, wind and pressure	~£130

A community node therefore costs less than a hundred pounds before sensors; the software costs nothing and needs no subscription. Our own Pi 5 and sensor came free in the competition’s starter kit, so those are replacement prices for anyone else building a node, not what we spent.

Twenty of those would be £1,700, which is not what we are proposing. The deployment shape is one Pi 5 as the hub — engine, model, database and dashboard — with leaf nodes that only read their channels and post the same canonical reading over Wi-Fi, which a £12 Pico W or ESP32 can do. A realistic school deployment is one hub and four leaves with basic sensors: about £150, or roughly £85 plus £15 a node after that. The canonical reading is what makes that substitution free: anything that can emit the eight channels with a source and a quality flag is a node, whatever it runs on.

Speed. The per-cycle cost is small: the 20 nodes are scored by the Isolation Forest in one batch rather than one at a time, which cut a full cycle from about 425 ms to about 23 ms on the x86-64 Linux machine used for the final review (four cores, 16 GB, Python 3.11), where the model trains in 0.2 s and the engine peaks at about 200 MB of memory.

On Arm. The same suite and benchmark run in our continuous integration on a real 64-bit Arm Linux machine (GitHub’s Arm runner: four Neoverse-N2 cores, 16 GB, Python 3.11 as the Bookworm release of Raspberry Pi OS ships). Every dependency installed from a prebuilt Arm wheel with compilation forbidden, all 212 tests passed in 13 s, the model trained in 0.1 s, a full 20-node cycle took 13–14 ms across runs and the engine peaked at 199 MB.

On the Pi. It runs on our own Pi 5; we have not put a stopwatch on it there, so for the board we give a bound rather than a number: its Cortex-A76 cores are slower than those server cores, but even if the Pi were eight times slower (two to three times is more likely) a cycle would take about 100 ms, 5 % of the 2-second read interval, and the 200 MB footprint sits comfortably inside the 4 GB board with the dashboard alongside. `python scripts/pi_benchmark.py` prints the measured sentence for whatever machine it runs on.

Installation. Every dependency ships a prebuilt 64-bit Arm wheel (the Arm job in our CI installs with compilation forbidden, to prove it), so installation on Raspberry Pi OS is one script (`setup_pi.sh`): it creates a Python environment, installs the requirements (no compiler or special tools needed) and runs the smoke test. The dashboard is only reachable from the Pi itself, or from a laptop connected to it securely over SSH, so nothing is opened up on the school network. The map has an offline basemap option, so even the Live Map works with no internet.

8. Impact

What it looks like on the day. It is a wet November morning. The water-level channel at the Regent’s Canal node climbs. Three neighbouring nodes report the same trend, so this is not one stuck sensor, and the site manager’s screen reads: *“Flood risk rising near Regent’s Canal (Little Venice). Same trend seen across 3 nearby nodes. Risk score 100/100. Main contributors: critical water level (4.8 m), very high humidity (99%), falling pressure (996 hPa).”* Under it are three things to do now: check and clear the nearby drains and gullies, inspect low-lying paths and entrances for standing water, and move valuables off the ground floor. The official area warning may follow later; the point is that someone on site had a reason to act before it did. (The quoted alert is the actual output of `python scripts/demo_tour.py` for the flood scenario, on simulated data.)

Climate Mesh is aimed at the people who actually respond first: a school site manager, a receptionist, a residents’ association. For the streets that official gauges do not cover, it gives them a risk score they can read at a glance, the reason it is rising, and three practical things to do. It stores only environmental measurements, never personal data, and runs entirely locally, so a school can adopt it without a data-protection review or a cloud bill. Because every node is cheap and every reading carries its provenance, a network of schools could pool their nodes into a genuine neighbourhood early-warning mesh.

9. Teamwork and what we learned

We are a team of two, Luis Yu and Leo Zhang. The first version of Climate Mesh went into our repository in early August 2026; by the end of that month it carried 144 automated tests, and the final review before submission brought that to 212. We worked on it together throughout, and we would both

be able to explain any part of it to a judge.

The first thing that worked was the simulator: twenty nodes with a daily cycle and some noise, and a risk score that went red when we typed in a flood. That took an afternoon and felt like most of the project. It was not. Almost everything after it was about trust: deciding what the system is allowed to claim. We argued about whether the demo could show a “sensor” reading for a sensor we did not have, and decided it could not. That decision is now enforced in code: a node can only be labelled **hardware** after a real device opens, and a badge saying “Physical Sensor” can only appear when a reading actually came from one. It cost us the easy screenshot and gave us the project’s spine.

The worst week was the one where we made the demo deterministic. Every screenshot and every number in this document had to be reproducible by a stranger, which meant fixed seeds, a frozen clock in judge mode, and tests that pin the published figures. We broke the numbers several times while “improving” the risk engine, and each time a test told us before a judge could. We also learned to test the dashboard itself, not just the maths: an automated run of all seven tabs found that one tab crashed whenever real data was present, which we had never noticed because we always looked at the first few tabs.

If we were starting again, or advising a Year 10 team: build the honest version first, make one command that proves it works, and only then make it look good.

What was hardest. The three things above: deciding what the system is allowed to claim (a reading is “hardware” only after a device physically opens; a water level derived from rainfall is flagged **estimated**; the AI records whether it trained on real or synthetic data), making every published number reproducible by a stranger, and testing the dashboard itself rather than just the maths. The crash the tab test caught was a whole column of data-source labels being tested as if it were a single yes/no value, which broke the Hardware Readiness tab whenever live data was present and blanked the Competition Pitch tab after it.

What we learned. The big one: an honest system is more work than an impressive one, and the work is mostly saying no to yourself. Technically: that anomaly detection finds odd *combinations* that thresholds miss; that a test which pins a number you have published is the best guard against quietly breaking your own claims; and that a whole machine-learning pipeline really does fit on a £55 computer (199 MB and about 14 ms a cycle on Arm).

FIGURE 7 — PHOTO GOES HERE

Luis and Leo with the Pi running the flood scenario

Replace this file: SUBMISSION_PACK/photos/photo-2-team.jpg

Figure 7 — ► PLACEHOLDER: replace photos/photo-2-team.jpg with a photo of the two of you at the Pi, and change the picture in WRITEUP.docx. Caption to use: "Luis and Leo testing the flood scenario on the Pi."

10. What we would do next

1. **Put it in front of somebody who is not us.** Give a site manager the same task we set ourselves in §6, watch without helping, and change whatever they struggle with. Our own walkthrough found four things; a stranger will find the ones we cannot see because we built it.
2. **Validate one physical node.** Read a real Vernier GDX-WTHR through the existing adapter and compare it against the digital twin for the same location.
3. **Calibrate against history.** Use the evidence export to compare risk scores with recorded local flood and heat events, and tune thresholds from data instead of by hand.
4. **Physical mesh links.** Two or more real Pi nodes exchanging readings (Wi-Fi first, LoRa later) so neighbour agreement runs across actual devices.
5. **Node cost pack.** A priced, tested bill of materials for a minimal community node, published once we have built one from it and run it.

11. Safety, privacy and responsible use

No personal data is stored: no names, accounts, cameras, microphones or tracking of people. The only optional outbound calls fetch public Open-Meteo weather data. Emergency scenarios are simulated and labelled as such, so judging never depends on or misrepresents a real emergency. Alerts are decision support and always defer to official emergency guidance. The hardware is low-voltage and USB-powered.

12. Acknowledgements and help received

Built in Python with Streamlit (dashboard), scikit-learn (Isolation Forest), pandas, NumPy, Plotly and SQLite. Live and archive weather data come from the free Open-Meteo service. The hardware path uses Vernier's `godirect` driver and `gdx` helper. Early-warning statistics are from the World Meteorological Organization and UNDRR. The heat index follows the NOAA/Steadman formulation.

Help received. We declare this whether or not the entry form asks, because the whole point of the project is saying where things come from.

Ours: the idea and the choice of problem, the architecture, the 20-node simulator, the six sub-scores and their thresholds, the neighbour-agreement rule, the provenance policy, the decision not to fake a sensor reading, and the 144 tests the project already had at the end of August.

Then, during the final review of this entry in September 2026, we used an AI coding assistant (Anthropic's Claude), working under our direction: it reviewed the code and fixed the bugs it found; added tests (the suite grew from 144 to 212); batched the anomaly scoring and wrote the benchmark script and the Arm CI job behind the figures in §7; added the offline basemap; captured the screenshots; replaced the licence text at our request; and drafted this document, the entry-form answers and the Word/PDF build from the repository's history and our notes, which we then read and corrected. The repository's September 2026 commits record that work. No teacher, mentor or other adult wrote any part of the code or of this document. The concept, the design and the original codebase are our own work.

Appendix — Running it yourself

```
python3 -m venv .venv && source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt
python scripts/judge_validate.py # smoke test + 212 tests + 2 demo cycles + export
python scripts/demo_tour.py # all five scenarios in a few seconds
python scripts/evaluate.py # detection vs false alarms, ~1 minute
python run.py --mode demo --scenario flood --judge-mode # terminal 1
python -m streamlit run dashboard/app.py # terminal 2 → http://localhost:8501
```

The repository's `README.md` has the full file map, run modes and troubleshooting; `docs/screenshots/` holds every dashboard tab and the terminal evidence.

The demo tour’s output, exactly as printed (the numbers quoted in §6):

```
=====
Climate Mesh - Demo Tour (deterministic, synthetic data, no hardware)
=====
Scenario      Avg   Max  Worst node      Level      Alerts
-----
normal        3.9  11.4  Greenwich        SAFE         0
flood         50.8 100.0  Regent's Canal (Little Venice) CRITICAL    10
heatwave      78.8 100.0  Brixton          CRITICAL    12
smog          82.7 100.0  Brixton          CRITICAL    15
storm         98.1 100.0  Brixton          CRITICAL    20
-----
Risk is 0-100. Every reading above is source=demo, is_simulated=True.
=====

Sample explanation (flood, Regent's Canal (Little Venice)):
  Flood risk rising near Regent's Canal (Little Venice). Same trend seen across 3 nearby nodes. Risk
score 100/100. Main contributors: critical water level (4.8 m), very high humidity (99%), falling
pressure (996 hPa).

RESULT: PASS (5/5 scenarios, deterministic, offline)
```